

实验名称 实验6 ER随机网络的生成及拓扑特征分析

一、实验内容及结果

- 1、利用 $G(N, M)$ 和 $G(N, p)$ 算法各生成 10 个网络 (N 固定, M 和 p 变化), 计算每个网络的平均度、平均路径长度、聚类系数、余平均度、余度分布
- 2、分析以上拓扑参数随着 M 或 p 的增加的变化趋势(分析)并作图

源代码:

定义网络:

$G(N, M)$

```
import networkx as nx
import matplotlib.pyplot as plt
import random
def GNM(N,M):
    G=nx.Graph()
    G.add_nodes_from(range(N))
    nlist=list(G)
    edge_count=0;
    while edge_count<M:
        u=random.choice(nlist)
        v=random.choice(nlist)
        if u==v or G.has_edge(u,v):
            continue
        else:
            G.add_edge(u,v)
            edge_count=edge_count+1
    return G
```

$G(N, p)$

```
import itertools
def GNP(N,p):
    edges=itertools.combinations(range(N),2)
    G=nx.Graph()
    G.add_nodes_from(range(N))
    for e in edges:
        if random.random()<p:
            G.add_edge(*e)
    return G
```

余平均度函数:

```
#余平均度
def average_nearest_neighbor_degree(G):
    k=set([G.degree(i) for i in G.nodes()])
    sorted_k=sorted(k)
    d=dict(nx.degree(G))
    avd=sum(d.values())/len(G.nodes)

    k_nn_k=[]
    for ki in sorted_k:
        if ki != 0:
            c=0
            k_nn_i=0
            for i in G.nodes():
                if G.degree(i)==ki:
                    k_nn_i+=sum([G.degree(j) for j in list(nx.all_neighbors(G,i))])/ki
                    c+=1
            k_nn_k.append(k_nn_i/c)

    m=sorted(k)
    p=k_nn_k
    edd = {}
    S=0
    for i in range(len(m)):
        edd[i] = p[i] * m[i] / avd
    for j in edd:
        S+=edd[j]
    a_edd=S/len(edd)
    return a_edd,edd,m
```

计算拓扑参数：

G(N,M):

```
G=[]
N=100
random.seed(1)
ML=[random.randint(0,N*(N-1)/2) for _ in range(10)]
ML=sorted(ML)
print(ML)
networks = []
for M in ML:
    G = nx.gnm_random_graph(N, M)
    networks.append(G)
AVD={}
L={}
AVT={}
AE={}

for i, G in enumerate(networks):
    d=dict(nx.degree(G))
    #平均度
    avd=sum(d.values())/len(G.nodes)
    AVD[i]=avd
    print(f"Graph {i+1}:")
    print(f"平均度: {avd}")

    #最短路径
    if nx.is_connected(G):
        l=nx.average_shortest_path_length(G)
    else:
        l=0
    L[i]=l
    print(f"最短路径: {l}")

    #聚类系数
    avt=nx.average_clustering(G)
    AVT[i]=avt
    print(f"聚类系数t: {avt}")

    #余平均度
    [ae,edd,k] = average_nearest_neighbor_degree(G)
    AE[i]=ae
    print(f"余平均度:{ae}")

    #余度分布
    print(f"余度分布:")
    for ki, ed in zip(k,edd.values()):
        print(f" Degree {ki}: {ed}")

    print()
```

G(N,P):

```
Gp=[]
N=100
random.seed(100)
P=[round(random.random(),2) for _ in range(10)]
P=sorted(P)

networks = []
for p in P:
    Gp = nx.erdos_renyi_graph(N, p)
    networks.append(Gp)

for i, Gp in enumerate(networks):
    d=dict(nx.degree(Gp))
    # 平均度
    avd=sum(d.values())/len(Gp.nodes)
    AVD[i]=avd
    print(f"Graph {i+1}:")
    print(f"平均度: {avd}")

    # 最短路径
    if nx.is_connected(Gp):
        l=nx.average_shortest_path_length(Gp)
    else:
        l=0
    L[i]=l
    print(f"最短路径: {l}")

    # 聚类系数
    avt=nx.average_clustering(Gp)
    AVT[i]=avt
    print(f"聚类系数t: {avt}")

    # 余平均度
    [ae,edd,k] = average_nearest_neighbor_degree(Gp)
    AE[i]=ae
    print(f"余平均度:{ae}")

print()
```

输出：

G(N,M)

```
for m, avd in zip(ML, AVD.values()):
    print(f"边数:{m} 平均度:{avd}")
for m, l in zip(ML, L.values()):
    print(f"边数:{m} 平均路径长度:{l}")
for m, avt in zip(ML, AVT.values()):
    print(f"边数:{m} 聚类系数:{avt}")
for m, ae in zip(ML, AE.values()):
    print(f"边数:{m} 余平均度:{ae}")
```

```
print(f"余度分布:")
for i, G in enumerate(networks):
    d = dict(nx.degree(G))
    ae, edd, k = get_pdf(G)

    print(f"M={ML[i]}:")
    for ki, ed in zip(k, edd.values()):
        print(f" Degree {ki}: {ed}")
```

G(N,P)

```
for p, avd in zip(P, AVD.values()):
    print(f"连边概率:{p} 平均度:{avd}")
for p, l in zip(P, L.values()):
    print(f"连边概率:{p} 平均路径长度:{l}")
for p, avt in zip(P, AVT.values()):
    print(f"连边概率:{p} 聚类系数:{avt}")
for p, ae in zip(P, AE.values()):
    print(f"连边概率:{p} 余平均度:{ae}")
```

```
#余度分布
print(f"余度分布:")
for i, Gp in enumerate(networks):
    print(f"P= {P[i]}:")
    for ki, ed in zip(k, edd.values()):
        print(f" Degree {ki}: {ed}")
```

绘图：

G(N,M)

```
plt.plot(ML, AVD.values(), marker='o', linestyle='-')
plt.xlabel('M')
plt.ylabel('AVD')
plt.title('<k> vs M')
plt.grid(True)
plt.show()
```

```
plt.plot(ML, L.values(), marker='o',
plt.xlabel('M')
plt.ylabel('L')
plt.title('L vs M')
plt.grid(True)
plt.show()
```

```
plt.plot(ML, AVT.values(), marker='o')
plt.xlabel('M')
plt.ylabel('AVT')
plt.title('C vs M')
plt.grid(True)
plt.show()
```

```
plt.plot(ML, AE.values(), marker='o',
plt.xlabel('M')
plt.ylabel('AE')
plt.title('AE vs M')
plt.grid(True)
plt.show()
```

G(N,P)

```
plt.plot(P, AVD.values(), marker='o', linestyle=
plt.xlabel('P')
plt.ylabel('AVD')
plt.title('<k> vs P')
plt.grid(True)
plt.show()
```

```
plt.plot(P, L.values(), marker='o', linestyle=
plt.xlabel('P')
plt.ylabel('L')
plt.title('L vs P')
plt.grid(True)
plt.show()
```

```
plt.plot(P, AVT.values(), marker='o', lines
plt.xlabel('P')
plt.ylabel('AVT')
plt.title('C vs P')
plt.grid(True)
plt.show()
```

```
plt.plot(P, AE.values(), marker='o', 1
plt.xlabel('P')
plt.ylabel('AE')
plt.title('AE vs P')
plt.grid(True)
plt.show()
```

```

plt.figure(figsize=(12, 8))
for i, Gp in enumerate(networks):
    d = dict(nx.degree(Gp))
    ae, edd, k = get_pdf(Gp)

    plt.plot(k, edd.values(), label=f'M= {ML[i]}', alpha=0.5)

plt.legend()
plt.xlabel('k')
plt.ylabel('AE')
plt.title('AE of 10 Networks')
plt.grid(True)
plt.show()

```

```

plt.figure(figsize=(12, 8))
for i, Gp in enumerate(networks):
    d = dict(nx.degree(Gp))
    ae, edd, k = get_pdf(Gp)

    plt.plot(k, edd.values(), label=f'P= {P[i]}', alpha=0.5)

plt.legend()

plt.xlabel('k')
plt.ylabel('AE')
plt.title('AE of 10 Networks')
plt.grid(True)

plt.show()

```

运行结果 (截图):

Task1:

G(N,M)

边数:516 平均度:10.32
边数:965 平均度:19.3
边数:1100 平均度:22.0
边数:1719 平均度:34.38
边数:2089 平均度:41.78
边数:3109 平均度:62.18
边数:3682 平均度:73.64
边数:3868 平均度:77.36
边数:4058 平均度:81.16
边数:4662 平均度:93.24

边数:516 平均路径长度:2.183838383838384
边数:965 平均路径长度:1.827070707070707
边数:1100 平均路径长度:1.7814141414141413
边数:1719 平均路径长度:1.6527272727272728
边数:2089 平均路径长度:1.577979797979798
边数:3109 平均路径长度:1.371919191919192
边数:3682 平均路径长度:1.2561616161616163
边数:3868 平均路径长度:1.2185858585858587
边数:4058 平均路径长度:1.1802020202020203
边数:4662 平均路径长度:1.0581818181818181

边数:516 聚类系数:0.09763572538572537
边数:965 聚类系数:0.19418075613607302
边数:1100 聚类系数:0.21147112008278066
边数:1719 聚类系数:0.3428598654337336
边数:2089 聚类系数:0.42218690409323173
边数:3109 聚类系数:0.6278898865771165
边数:3682 聚类系数:0.7438122435946484
边数:3868 聚类系数:0.7817144222447976
边数:4058 聚类系数:0.818806356709921
边数:4662 聚类系数:0.9418041939907268

边数:516 余平均度:11.061578667849068
边数:965 余平均度:21.329751894302845
边数:1100 余平均度:22.74877001653317
边数:1719 余平均度:35.022558823278715
边数:2089 余平均度:42.96062646110516
边数:3109 余平均度:62.39675275940955
边数:3682 余平均度:73.86668503805929
边数:3868 余平均度:77.08567349603608
边数:4058 余平均度:81.03681081983203
边数:4662 余平均度:92.6082035500803

G(N,p)

连边概率:0.08 平均度:7.98
连边概率:0.15 平均度:15.34
连边概率:0.43 平均度:43.12
连边概率:0.45 平均度:45.18
连边概率:0.46 平均度:44.88
连边概率:0.53 平均度:52.32
连边概率:0.71 平均度:70.16
连边概率:0.73 平均度:72.12
连边概率:0.77 平均度:76.32
连边概率:0.8 平均度:79.74

连边概率:0.08 平均路径长度:2.421818181818182
连边概率:0.15 平均路径长度:1.9131313131313132
连边概率:0.43 平均路径长度:1.5644444444444445
连边概率:0.45 平均路径长度:1.5436363636363637
连边概率:0.46 平均路径长度:1.5466666666666666
连边概率:0.53 平均路径长度:1.4715151515151514
连边概率:0.71 平均路径长度:1.2913131313131314
连边概率:0.73 平均路径长度:1.2715151515151515
连边概率:0.77 平均路径长度:1.229090909090909
连边概率:0.8 平均路径长度:1.1945454545454546

连边概率:0.08 聚类系数:0.07975235875235871
连边概率:0.15 聚类系数:0.1444298464058189
连边概率:0.43 聚类系数:0.43745055249822884
连边概率:0.45 聚类系数:0.4589471820406958
连边概率:0.46 聚类系数:0.4525218051335774
连边概率:0.53 聚类系数:0.5281064477825216
连边概率:0.71 聚类系数:0.7129056449107207
连边概率:0.73 聚类系数:0.7278108079912363
连边概率:0.77 聚类系数:0.7701576655298176
连边概率:0.8 聚类系数:0.8060792930387529

连边概率:0.08 余平均度:8.695593329889089
连边概率:0.15 余平均度:17.21648211360565
连边概率:0.43 余平均度:45.70783799179846
连边概率:0.45 余平均度:45.1078239878594
连边概率:0.46 余平均度:46.37466628860344
连边概率:0.53 余平均度:53.579894049872756
连边概率:0.71 余平均度:69.39493131346039
连边概率:0.73 余平均度:72.14023689284218
连边概率:0.77 余平均度:76.74364407527406
连边概率:0.8 余平均度:79.41491841048926

余度分布:

M=516:

Degree 2: 0.005012531328320802
Degree 3: 0.007518796992481203
Degree 4: 0.015037593984962405
Degree 5: 0.05012531328320802
Degree 6: 0.08270676691729323
Degree 7: 0.16666666666666666
Degree 8: 0.15037593984962405
Degree 9: 0.14661654135338345
Degree 10: 0.15037593984962405
Degree 11: 0.08270676691729321
Degree 12: 0.07518796992481204
Degree 13: 0.03258145363408521
Degree 14: 0.03508771929824561

M=965:

Degree 9: 0.02346805736636245
Degree 10: 0.00651890482398957
Degree 11: 0.04302477183833116
Degree 12: 0.07040417209908736
Degree 13: 0.1271186440677966
Degree 14: 0.11864406779661017
Degree 15: 0.07822685788787484
Degree 16: 0.10430247718383312
Degree 17: 0.08865710560625815
Degree 18: 0.08213820078226859
Degree 19: 0.06192959582790092
Degree 20: 0.05215123859191656
Degree 21: 0.054758800521512385
Degree 22: 0.04302477183833116
Degree 23: 0.02998696219035202
Degree 24: 0.01564537157757497

M=1719:

Degree 30: 0.006640106241699867
Degree 31: 0.006861443116423196
Degree 37: 0.01637892872952634
Degree 38: 0.033643204957945996
Degree 39: 0.03452855245683931
Degree 40: 0.03541389995573263
Degree 41: 0.03629924745462594
Degree 42: 0.055776892430278883
Degree 43: 0.12372731297034086
Degree 44: 0.029216467463479414
Degree 45: 0.0896414342629482
Degree 46: 0.04072598494909252
Degree 47: 0.11443116423196105
Degree 48: 0.11686586985391767
Degree 49: 0.10845506861443117
Degree 50: 0.033200531208499334
Degree 51: 0.02257636122177955
Degree 52: 0.023019034971226208
Degree 53: 0.023461708720672866
Degree 54: 0.023904382470119525
Degree 57: 0.025232403718459497

余度分布:

P= 0.08:

Degree 63: 0.007900677200902935
Degree 69: 0.008653122648607977
Degree 71: 0.00890393779784299
Degree 72: 0.009029345372460498
Degree 73: 0.018309505894156006
Degree 74: 0.01856032104339102
Degree 75: 0.05643340857787811
Degree 76: 0.0762478053674442
Degree 77: 0.04828191622774016
Degree 78: 0.08803611738148984
Degree 79: 0.05944319036869827
Degree 80: 0.11035866566340609
Degree 81: 0.10158013544018059
Degree 82: 0.13368447454226237
Degree 83: 0.08327062954602459
Degree 84: 0.09480812641083522
Degree 85: 0.021319287684976172
Degree 86: 0.010785051417105593
Degree 87: 0.0109104589917231
Degree 88: 0.011035866566340608
Degree 89: 0.011161274140958116
Degree 90: 0.011286681715575621

P= 0.15:

Degree 63: 0.007900677200902935
Degree 69: 0.008653122648607977
Degree 71: 0.00890393779784299
Degree 72: 0.009029345372460498
Degree 73: 0.018309505894156006
Degree 74: 0.01856032104339102
Degree 75: 0.05643340857787811
Degree 76: 0.0762478053674442
Degree 77: 0.04828191622774016
Degree 78: 0.08803611738148984
Degree 79: 0.05944319036869827
Degree 80: 0.11035866566340609
Degree 81: 0.10158013544018059
Degree 82: 0.13368447454226237
Degree 83: 0.08327062954602459
Degree 84: 0.09480812641083522
Degree 85: 0.021319287684976172
Degree 86: 0.010785051417105593
Degree 87: 0.0109104589917231
Degree 88: 0.011035866566340608
Degree 89: 0.011161274140958116
Degree 90: 0.011286681715575621

M=2089:

Degree 35: 0.007798573975044564
Degree 36: 0.02406417112299465
Degree 37: 0.008244206773618538
Degree 38: 0.0338680926916221
Degree 39: 0.034759358288770054
Degree 40: 0.008912655971479501
Degree 41: 0.07308377896613191
Degree 42: 0.08422459893048127
Degree 43: 0.05748663101604278
Degree 44: 0.06862745098039215
Degree 45: 0.1303475935828877
Degree 46: 0.14349376114081996
Degree 47: 0.0733065953654189
Degree 48: 0.0320855614973262
Degree 49: 0.05459001782531194
Degree 50: 0.03342245989304812
Degree 52: 0.057932263814616754
Degree 53: 0.023618538324420676
Degree 54: 0.012032085561497326
Degree 55: 0.012254901960784314
Degree 57: 0.01270053475935829
Degree 59: 0.013146167557932263

...

...

...

M=4662:

Degree 63: 0.007900677200902935
Degree 69: 0.008653122648607977
Degree 71: 0.00890393779784299
Degree 72: 0.009029345372460498
Degree 73: 0.018309505894156006
Degree 74: 0.01856032104339102
Degree 75: 0.05643340857787811
Degree 76: 0.0762478053674442
Degree 77: 0.04828191622774016
Degree 78: 0.08803611738148984
Degree 79: 0.05944319036869827
Degree 80: 0.11035866566340609
Degree 81: 0.10158013544018059
Degree 82: 0.13368447454226237
Degree 83: 0.08327062954602459
Degree 84: 0.09480812641083522
Degree 85: 0.021319287684976172
Degree 86: 0.010785051417105593
Degree 87: 0.0109104589917231
Degree 88: 0.011035866566340608
Degree 89: 0.011161274140958116
Degree 90: 0.011286681715575621

P= 0.43:

Degree 63: 0.007900677200902935
Degree 69: 0.008653122648607977
Degree 71: 0.00890393779784299
Degree 72: 0.009029345372460498
Degree 73: 0.018309505894156006
Degree 74: 0.01856032104339102
Degree 75: 0.05643340857787811
Degree 76: 0.0762478053674442
Degree 77: 0.04828191622774016
Degree 78: 0.08803611738148984
Degree 79: 0.05944319036869827
Degree 80: 0.11035866566340609
Degree 81: 0.10158013544018059
Degree 82: 0.13368447454226237
Degree 83: 0.08327062954602459
Degree 84: 0.09480812641083522
Degree 85: 0.021319287684976172
Degree 86: 0.010785051417105593
Degree 87: 0.0109104589917231
Degree 88: 0.011035866566340608
Degree 89: 0.011161274140958116
Degree 90: 0.011286681715575621

...

...

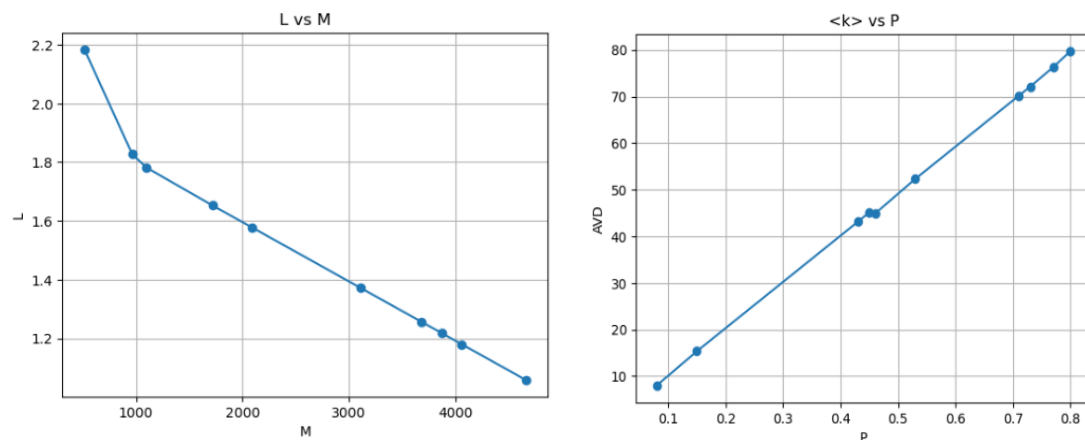
...

P= 0.8:

Degree 63: 0.007900677200902935
Degree 69: 0.008653122648607977
Degree 71: 0.00890393779784299
Degree 72: 0.009029345372460498
Degree 73: 0.018309505894156006
Degree 74: 0.01856032104339102
Degree 75: 0.05643340857787811
Degree 76: 0.0762478053674442
Degree 77: 0.04828191622774016
Degree 78: 0.08803611738148984
Degree 79: 0.05944319036869827
Degree 80: 0.11035866566340609
Degree 81: 0.10158013544018059
Degree 82: 0.13368447454226237
Degree 83: 0.08327062954602459
Degree 84: 0.09480812641083522
Degree 85: 0.021319287684976172
Degree 86: 0.010785051417105593
Degree 87: 0.0109104589917231
Degree 88: 0.011035866566340608
Degree 89: 0.011161274140958116
Degree 90: 0.011286681715575621

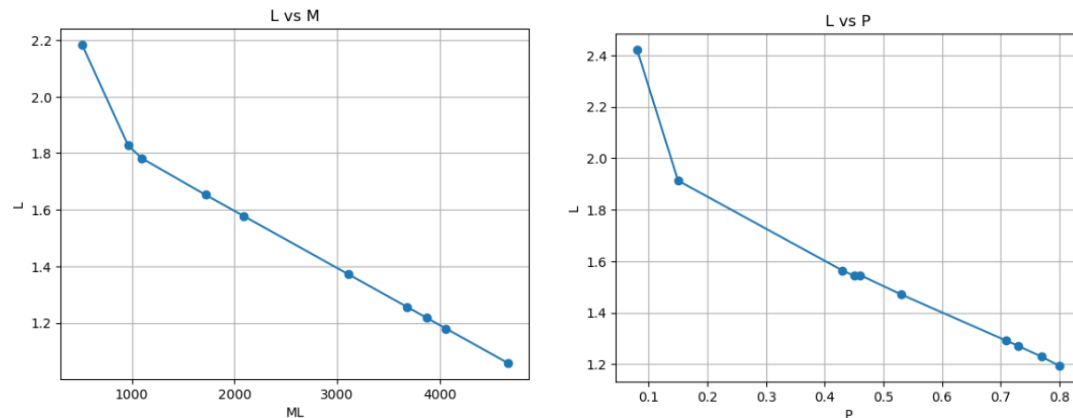
TASK2:

平均度:



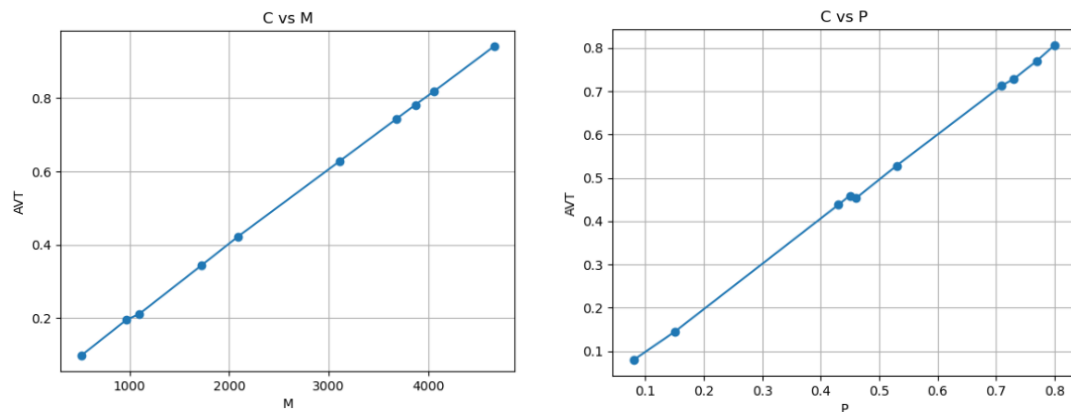
分析：在节点 N 固定的情况下，网络平均度 $\langle k \rangle$ 随边数 M 与连边概率 P 的增加线性增加。因为 $\langle k \rangle = 2M/N$ ， $\langle k \rangle = N \cdot P$ ，平均度与 M 和 P 均成正比。

平均路径长度:



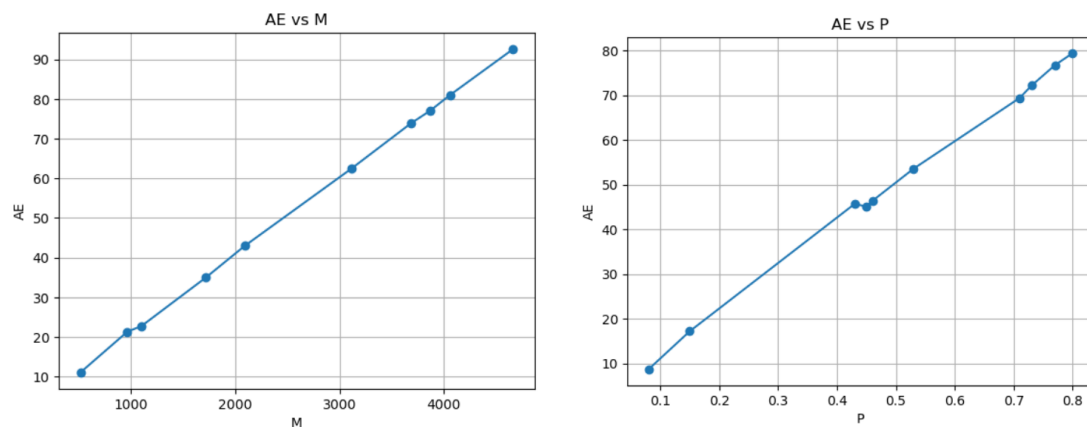
分析：在节点 N 固定的情况下，平均路径长度 L 随边数 M 与连边概率 P 的增加呈减小趋势，且减小速率由高到低。因为当网络较为稀疏时，节点间的连边比较少，平均路径长度较长，随着边数与连边概率的增加，节点间连接变得更加密集，平均路径长度减小；且当网络足够密集时，大多数节点已互相连接，新增加的边对平均路径的长度影响减弱，减小速率变慢。

聚类系数：



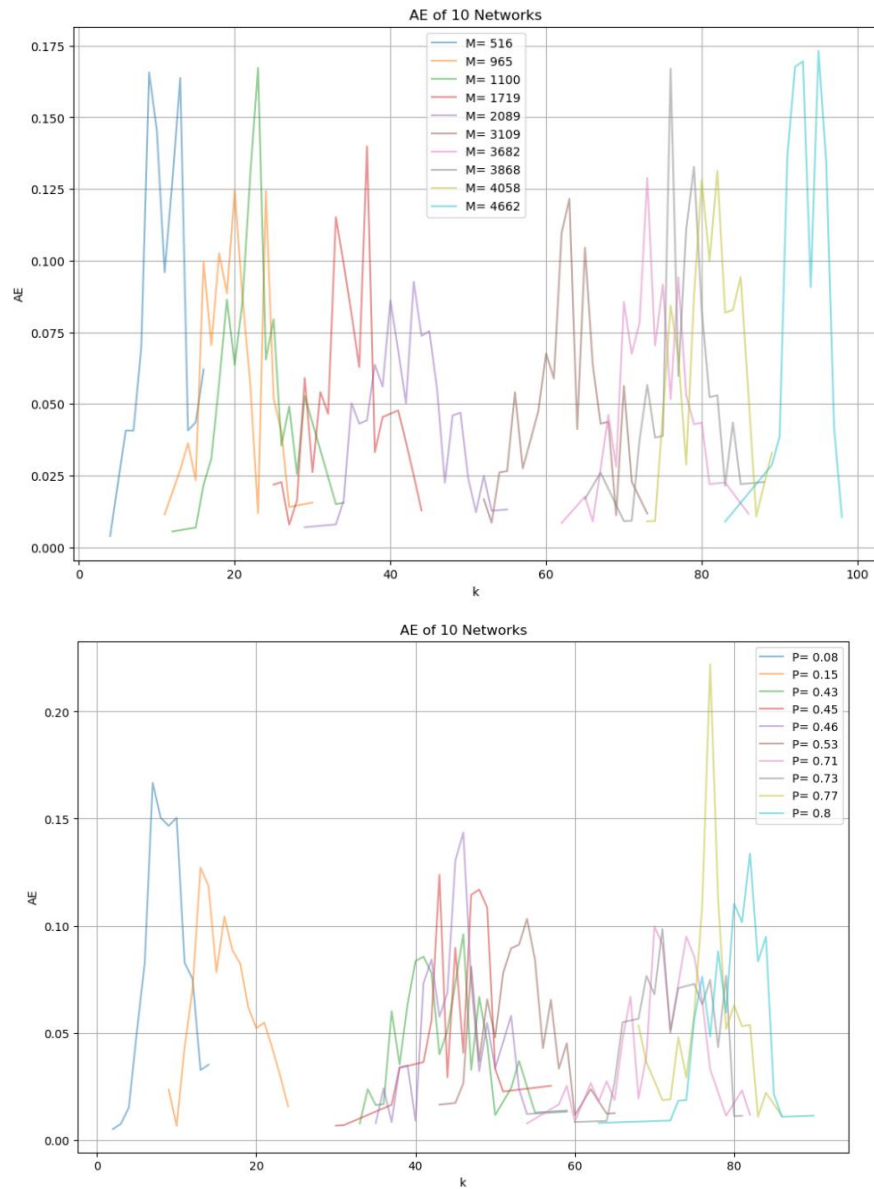
分析：在节点 N 固定的情况下，聚类系数 C 随边数 M 与连边概率 P 的增加而线性增加，且在 E-R 随机图中，满足 $C=P$ 。因为对于 $G(N, M)$ ，
$$C = \frac{\langle k \rangle}{N-1} = \frac{2M}{N(N-1)}$$
， C 与 M 线性相关；而 $G(N, p)$ 的两个节点间无论是否具有共同的邻居节点，连接概率均为 p ，即 $C=P$ 。

余平均度：



分析：在节点 N 固定的情况下，余平均度随着边数 M 与连边概率 P 的增加呈现递增趋势。因为当网络变得更密集时，每个节点的邻居节点数量增加，平均邻居度提高，即余平均度增加。

余度分布：



分析：在节点 N 固定的情况下，余度分布随边数 M 与连边概率 P 的增加逐渐展现出更大的 k 值，即整体呈现向右偏移趋势。因为网络中随机选取的一个节点的邻居节点的平均度随着网络变密集而增大。